

Repolens

An Agentic AI Repository Auditor

14-day solo build documenting product, engineering, and distribution decisions.

Haichen Duan

TUM Wirtschaftsinformatik (BSc), May 2026

Live deployment

Web UI: <https://repolens-audit.vercel.app>

API: <https://repolens-production-61e0.up.railway.app>

Source: <https://github.com/Haichennn/repolens>

Executive summary

Repolens is an agentic AI system that audits any GitHub repository across five dimensions (documentation, architecture, maintenance, testing, security) in approximately 30 to 70 seconds. It is built with LangChain, LangGraph, the Model Context Protocol, FastAPI, and Next.js, fully deployed across Railway and Vercel with CI green on every push.

The system extends beyond a single audit into three composed views: structured findings per dimension, a decision memo with an adopt or pass verdict, and a per-dependency due diligence report covering both Python (PyPI) and JavaScript (npm) ecosystems.

Repolens ships across three distribution channels: a web UI for interactive demos, a public REST API for programmatic access, and a Claude Skill that lets any Skill-aware editor (Cursor, Claude Code) invoke Repolens from inside the user's existing tooling.

This case study walks through three engineering decisions that defined the project (MCP integration, SSE streaming, Skill as distribution), the architecture choices, and the reflections that shaped the V3 roadmap.

Origin story

Two motivations converged into Repolens.

1. As a builder

As I started developing my own projects, GitHub repositories felt overwhelming. I did not know what industry-standard quality looked like, what good documentation or architecture should resemble. I needed a way to evaluate the repos I was learning from. The spark came when I used a frontend-design skill to audit my own portfolio (joanduan.dev) and realized this evaluative lens should exist for entire repos, not just UI surfaces.

2. As a Claude user

I often want Claude to review a complete GitHub repository: its structure, its tradeoffs, its quality. But Claude cannot autonomously clone and audit a whole repo. I can describe what I implemented and Claude can comment, but the comprehensive overview, scoring, and structured findings have to come from a separate system. Repolens fills that gap: it is the audit infrastructure Claude lacks, exposed both as a website and as a Skill that Claude can call from inside Cursor or Claude Code.

Why these two motivations matter together

These two motivations made Repolens not just "a tool I built" but "the tool I needed." The product validates against my own workflow first. The Skill channel solves the same gap for any Claude user. The web UI demonstrates the system end-to-end for stakeholders who want to see it work without integrating.

The dogfooding loop has run continuously: Repolens audited itself early in the build at 67 of 100. I fixed the flagged issues (missing LICENSE, no CONTRIBUTING.md, unpinned dependencies, no CI pipeline). The re-audit pushed it to 80-plus. The product validated the product.

Product overview

Five-dimensional audit

Each dimension produces a score from 0 to 100, a severity label (good, warning, critical), a one-sentence summary, evidence-cited findings, and prioritized recommendations.

- Documentation: README completeness, audience-fit, doc hints
- Architecture: file organization, framework choices, configuration hygiene
- Maintenance: commit recency, contributor health, release cadence
- Testing: test-to-source ratio, framework detection, CI presence
- Security: CVE counts, dependency vulnerabilities via MCP server

Composed deliverables

The single audit becomes the input to three follow-on views:

- Decision memo: 200-word adopt or pass verdict for stakeholders
- Due diligence: per-package risk + license + alternatives across PyPI and npm
- Comparison: side-by-side ranking of multiple repos with per-dimension winners

Three distribution channels

- Web UI (Vercel): interactive demo with live SSE streaming
- REST API (Railway): programmatic access for tooling integration
- Claude Skill: invocation from inside Cursor, Claude Code, or any Skill-aware editor

Tech stack

Backend: Python 3.12, FastAPI, LangChain, LangGraph, MCP, Anthropic SDK, Pydantic v2, PyGithub.
Frontend: Next.js 16 App Router, TypeScript strict, Tailwind v4, shadcn/ui. Infrastructure: Railway (backend Docker), Vercel (frontend), GitHub Actions CI.

Decision 1: MCP integration for the Security audit

Why

The Security audit needs to query CVE databases and package registries. I could call those APIs directly from the audit node, but I specifically wanted to demonstrate familiarity with the Model Context Protocol, Anthropic's late-2024 standard for giving Claude structured tool access. MCP is the senior pattern for tool-using agents in the post-2024 stack.

What

I built a custom MCP server under `backend/mcp_server/` that exposes two tools to Claude: `lookup_cves(package_name)` returns CVE entries with severity, CVSS score, affected versions, and fixed versions. `lookup_package(package_name)` returns registry metadata including license, weekly downloads, last publish date, and deprecation status. The server runs as a Python subprocess and communicates over JSON-RPC via stdio, the standard MCP transport.

How

The Security audit node spawns the MCP subprocess at audit time, queries it for each parsed dependency, then synthesizes findings with Claude. The MCP client session uses the official Python SDK:

```
server_params = StdioServerParameters(
    command=sys.executable,
    args=["-m", "mcp_server.server"],
    env=None,
)
async with stdio_client(server_params) as (read, write):
    async with ClientSession(read, write) as session:
        await session.initialize()
        result = await session.call_tool(
            "lookup_cves", {"package_name": pkg}
        )
```

What broke

A bug surfaced on Day 11 during local verification: my initial implementation used `command="python"`, which failed on macOS where only `python3` is symlinked in the venv. The fix was a one-line change to `command=sys.executable`, which uses whichever interpreter is actually running the parent process. This made the spawn portable across local development, Docker, and Railway production. The same fix could have been missed until production breakage if I hadn't been dogfooding.

Signal

This decision demonstrates familiarity with very recent 2024 Q4 tooling, JSON-RPC protocol mechanics, subprocess management, and the discipline to fix portability issues during development. Most junior implementations would call PyPI and CVE APIs directly. MCP integration is the pattern senior engineers reach for when building agent infrastructure that other Claude sessions can also consume.

Decision 2: SSE streaming refactor

Why

V1 (shipped Day 8) returned the full audit report after roughly 30 seconds. The user saw a blank screen for the wait. After shipping, I noticed something missing: the agentic narrative ("five agents working in parallel") was completely invisible. Users could not see the parallelism that was the whole point of the system. The UX told the wrong story about the engineering.

What

I refactored the LangGraph orchestration from `.invoke()` to `.astream()`, added a FastAPI `/audit/stream` endpoint that emits Server-Sent Events as each node completes, and replaced the frontend fetch call with EventSource plus per-dimension state updates.

How (three phases)

Phase 1: Backend graph. Added `audit_repo_streaming`, an async generator that yields one event per node completion:

```
{
  "type": "node_complete",
  "node": "documentation",
  "data": {
    "score": 72,
    "severity": "good",
    "findings": [...]
  }
}
```

Phase 2: FastAPI endpoint. Wrapped the generator in `StreamingResponse` with media type `text/event-stream`, proper SSE framing (`event: TYPE` then `data: JSON`), plus `Cache-Control` and `X-Accel-Buffering` headers for proxy resistance.

Phase 3: Frontend EventSource. Switched the React component from `fetch` to `EventSource` with `addEventListener("node_complete", ...)` for typed events. State updates flow per-dimension so finished rows render their score and severity color while siblings still show the scanning animation.

What broke

The initial EventSource implementation kept the default auto-retry on disconnect. Caught in design review: a transient network blip would have caused the backend to re-run a full audit, wasting compute and double-charging the Anthropic API. The fix was to call `eventSource.close()` manually inside `onerror` and expose a "Connection lost" error state in the UI instead. Distinguish the backend-emitted `event: error` frame from the browser-emitted connection error, since EventSource handles them via different code paths.

Signal

Self-identified UX gap after V1 shipped, then shipped a real infrastructure upgrade across three layers (graph orchestration, HTTP transport, frontend state). This is post-MVP quality work that's hard to fake. It means I'm not shipping for compile, I'm shipping for users.

Decision 3: Skill as distribution channel

Why

My V2 roadmap originally had "Comparative Ranking" plus an in-product "built-in terminal" where users could fix flagged issues directly inside Repolens. Within hours of starting that build, I realized something obvious in retrospect: users already have Cursor, Claude Code, and Codex. Building a worse version-of-their-IDE inside Repolens is repeating the wheel.

The instinct flipped. If users have a Claude-aware IDE, Repolens should plug into their IDE, not compete with it.

What

I shipped Repolens as a Claude Skill at `skills/repolens/SKILL.md`. A user installs the Skill once, then can audit any repo from inside their existing editor by saying things like: "audit github.com/owner/repo" or "compare langchain and llama-index for RAG." Claude reads the SKILL.md instructions, calls the Repolens API with the right endpoint, parses the JSON, and presents findings in the user's current context.

How

SKILL.md is just markdown with YAML frontmatter, Anthropic's Skill spec. No new code, no compute artifacts. The file lists endpoints (`/audit`, `/audit/stream`, `/memo`, `/due-diligence`, `parallel comparison`), maps natural-language patterns ("should we adopt", "compare these libraries") to which endpoint Claude should call, and gives presentation guidelines (lead with the score, cite specific dimensions, no generic PM-speak).

Total install footprint: two files (SKILL.md + a setup README), 127 lines for SKILL.md.

Insight

The mental shift from "feature" to "distribution channel" reframes what shipping a V2 product means in the AI era. Code is not the only artifact. Instructions to other AIs are. Repolens now exists in three channels: website (demo and marketing), API (programmatic), Skill (daily-driver inside the user's IDE).

Signal

Product instinct, willingness to abandon a feature mid-build when a better pattern emerges, and forward-thinking about AI-native distribution. The Skill channel is also the cheapest to maintain: as long as the API stays stable, the Skill keeps working with no code changes.

Product walkthrough

Tracing a typical user flow on the deployed web UI:

1. Submit a repo

The hero page presents a clean editorial-mono layout with a single input: "paste any github repo url." The user pastes a URL and clicks "run audit."

2. Watch the audit stream

Five dimension rows populate progressively over the next 30 to 70 seconds. The first row (typically documentation) lands at roughly 14 seconds. The slowest (architecture, with file-tree analysis) lands around 30 seconds. Each row has a severity-encoded color: sage green for good, burnt orange for warning, red for critical.

3. Drill into a dimension

Click any row to expand findings and recommendations. Findings are evidence-cited ("Documentation score 72/100: missing setup instructions, no CONTRIBUTING.md"). Recommendations are prioritized and actionable.

4. Generate a decision memo

Switch to the Memo tab, click "generate decision memo." The result is a three-tier verdict (adopt, adopt with caution, pass) plus strengths, concerns, next steps if adopting, and red flags to monitor. Roughly 200 words total, scannable in 30 seconds.

5. Run dependency due diligence

Switch to the Due Diligence tab, click generate. The system parses manifests (requirements.txt, pyproject.toml, package.json), queries PyPI and npm registries in parallel for each dependency, then synthesizes per-package risk levels with Claude. Output includes risk factors, license compatibility, abandoned packages, and alternative suggestions for high-risk dependencies.

6. Compare multiple repos

Click "+ add another repo to compare" and submit a second URL. The view morphs into a side-by-side comparison table with scores per dimension, best/worst pips, and a ranking section that identifies the winner per category and overall.

Engineering stack and architecture

Backend

FastAPI exposes both sync and async endpoints (`/audit` runs sync to avoid event-loop conflicts with the Security node's `asyncio.run`; `/audit/stream` runs async for SSE). LangGraph orchestrates the audit as a state-passing graph: one `fetch_repo` node fans out to five parallel audit nodes, then fans in to an aggregate node that computes a weighted overall score. The MCP server runs as a subprocess via JSON-RPC stdio. Pydantic v2 provides structured output guarantees on every LLM call.

Frontend

Next.js 16 App Router with TypeScript in strict mode. Tailwind v4 (OKLCH color encoding for severity). shadcn/ui with the Nova preset for input and button primitives. EventSource for SSE streaming, with manual `close()` to suppress auto-retry. Custom components for the audit dashboard, memo card, due-diligence card, and comparison view (multi-column grid with collapsible drill-downs).

Infrastructure

Backend deployed to Railway via a multi-stage Dockerfile with uvicorn running as PID 1 (`sh -c exec`, so `SIGTERM` goes straight to uvicorn for graceful shutdown). Frontend deployed to Vercel via Next.js auto-detection. GitHub Actions CI runs on every push: parallel backend job (Python syntax check, schema imports, graph and API import smoke tests) and frontend job (ESLint, `tsc --noEmit`, `next build`).

Code quality artifacts

All dependencies pinned in `requirements.txt` and `package-lock.json`. MIT license, `CONTRIBUTING.md`, comprehensive `.env.example` with source URLs. Monorepo structure (`backend/` + `frontend/` + `skills/`). Semantic git tags: `v0.1.0` (backend ship, Day 7), `v0.2.0` (full-stack live with SSE, Day 11).

Reflections and V3 roadmap

What I learned

- The hardest engineering decision is not what to build but when to redirect to existing infrastructure instead of building it yourself. Skill over Built-in Terminal was the cleanest version of that decision.
- Self-flagging UX issues post-MVP (Day 10 SSE refactor) is the discipline that separates "ship a prototype" from "ship a product." A working V1 is the start of the conversation, not the end.
- AI-assisted development is a partnership. I make product decisions, define quality bars, and pivot when needed. Claude Code executes the typing. The skill is not in writing code; it is in knowing what to build and what not to build.
- The dogfooding loop is the truest validation. Repolens audited itself, flagged its own missing infrastructure, I fixed the gaps, re-audit pushed the score up. The product gets credibility by being the first thing it improves.

Mid-build feedback that shaped V3

While building, I (as the primary user) flagged that the highest-value use case is not auditing a remote GitHub URL. It is auditing your own work-in-progress before submitting to a manager or mentor. The current architecture is GitHub-API-driven and does not support this. V3 will introduce local repo upload as the next infrastructure step.

V3 roadmap

- **Local repo audit (pre-push):** upload a folder or zip, audit without GitHub push. The natural next infrastructure step.
- **Audit history and score trajectory:** track repos over time as users fix issues. Persistence layer required.
- **Repolens as installable Skill in package manager:** one-command install for daily-driver distribution.
- **Multi-ecosystem dependency support:** Go (go.mod), Rust (Cargo.toml), Java (pom.xml, build.gradle).
- **Real-time SSE for memo and due-diligence generation:** same progressive-reveal pattern, applied beyond the initial audit.

Contact and links

Web demo: <https://repolens-audit.vercel.app>

API: <https://repolens-production-61e0.up.railway.app>

GitHub: <https://github.com/Haichennn/repolens>

Portfolio: <https://joanduan.dev>